

Dynamic Programming

- 1) Naïve Recursive Solution
- 2) Top-Down DP- Memoization {uses recursion}
- 3) Bottom-Up DP- Tabulation {uses loops}
- 4) $\rightarrow$ (if possible) Constant Space

Explained with LeetCode QNo. 509 Fibonacci Number

$$F(0) = 0 \quad F(1) = 1$$

$$F(n) = F(n-1) + F(n-2), \text{ for } n > 1$$

class Solution:

#Recursive Solution

```
def fib(self, n: int) -> int:
```

```
    if n == 0:
```

```
        return 0
```

```
    if n == 1:
```

```
        return 1
```

```
    return self.fib(n-2) + self.fib(n-1)
```

Top-Down DP- Memoization Calculate one, use it later

map the function to that function call's result in a dictionary / hashmap / cache / memo (same thing different names)

class Solution:

```
def fib(self, n: int) -> int:
```

```
    memo = {0: 0, 1: 1}
```

```
    def f(x):
```

```
        if x in memo:
```

```
            return x
```

```
        else:
```

```
            return memo[x] = f(x-1) + f(x-2)
```

```
            return memo[x]
```

```
    return f(n)
```

{0: 0, 1: 1, 2: 1, 3: 2, 4: 3, 5: 5, 6: 8}

Time: $O(n)$

Space: $O(n)$

Bottom-Up DP-Tabulation $n=8$ Create an empty array of $(n+1)$ size
$$\Rightarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

0 1 2 3 4 5 6 7 8

Define Base cases

$$\Rightarrow \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

0 1 2 3 4 5 6 7 8

 $dp[i] = dp[i-2] + dp[i-1]$

$$\Rightarrow \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

0 1 2 3 4 5 6 7 8

repeat

$$\Rightarrow \begin{bmatrix} 0 & 1 & 1 & 2 & 3 & 5 & 8 & 13 & 21 \end{bmatrix}$$

0 1 2 3 4 5 6 7 8

return the final element

class solution:

def fib(self, n: int) -> int:

if $n==0$:
return 0if $n==1$:
return 1Time: $O(n)$ Space: $O(n)$ Optimizedprev = 0
cur = 1

for i in range(2, n+1):

prev, cur = cur, prev + cur

return cur

dp = [0] * (n+1)

dp[0] = 0

dp[1] = 1

for i in range(2, n+1):

dp[i] = dp[i-2] + dp[i-1]

return dp[n]

Time: $O(n)$ Space: $O(1)$